

```
In [27]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
In [28]: df = pd.read_csv("Housing.csv")
```

```
In [29]: df.head()
```

```
Out[29]:
```

	price	area	bedrooms	bathrooms	stories	mainroad	guestroom	basement	hotw
0	13300000	7420	4	2	3	yes	no	no	
1	12250000	8960	4	4	4	yes	no	no	
2	12250000	9960	3	2	2	yes	no	yes	
3	12215000	7500	4	2	2	yes	no	yes	
4	11410000	7420	4	1	2	yes	yes	yes	

```
In [30]: df.isnull().sum()
```

```
Out[30]: price          0
area          0
bedrooms       0
bathrooms      0
stories        0
mainroad       0
guestroom      0
basement       0
hotwaterheating 0
airconditioning 0
parking        0
prefarea       0
furnishingstatus 0
dtype: int64
```

```
In [31]: df.columns
```

```
Out[31]: Index(['price', 'area', 'bedrooms', 'bathrooms', 'stories', 'mainroad',
               'guestroom', 'basement', 'hotwaterheating', 'airconditioning',
               'parking', 'prefarea', 'furnishingstatus'],
              dtype='object')
```

```
In [32]: binary_cols = ['mainroad', 'guestroom', 'basement',
                        'hotwaterheating', 'airconditioning', 'prefarea']

df[binary_cols] = df[binary_cols].replace({'yes': 1, 'no': 0})

df = pd.get_dummies(df, columns=['furnishingstatus'], drop_first=True)
```

```
C:\Users\System05\AppData\Local\Temp\ipykernel_2116\1918252449.py:4: FutureWarning:
Downcasting behavior in `replace` is deprecated and will be removed in a future vers
ion. To retain the old behavior, explicitly call `result.infer_objects(copy=False)`.
To opt-in to the future behavior, set `pd.set_option('future.no_silent_downcasting',
True)`
df[binary_cols] = df[binary_cols].replace({'yes': 1, 'no': 0})
```

In [33]: `df.head()`

Out[33]:

	price	area	bedrooms	bathrooms	stories	mainroad	guestroom	basement	hotw
--	-------	------	----------	-----------	---------	----------	-----------	----------	------

0	13300000	7420	4	2	3	1	0	0
1	12250000	8960	4	4	4	1	0	0
2	12250000	9960	3	2	2	1	0	1
3	12215000	7500	4	2	2	1	0	1
4	11410000	7420	4	1	2	1	1	1

In [37]: `from sklearn.preprocessing import LabelEncoder`
`le = LabelEncoder()`
`df['furnishingstatus_semi-furnished'] = le.fit_transform(df['furnishingstatus_semi-`
`df['furnishingstatus_unfurnished'] = le.fit_transform(df['furnishingstatus_unfurnis`

In [39]: `df.head()`

Out[39]:

	price	area	bedrooms	bathrooms	stories	mainroad	guestroom	basement	hotw
--	-------	------	----------	-----------	---------	----------	-----------	----------	------

0	13300000	7420	4	2	3	1	0	0
1	12250000	8960	4	4	4	1	0	0
2	12250000	9960	3	2	2	1	0	1
3	12215000	7500	4	2	2	1	0	1
4	11410000	7420	4	1	2	1	1	1

In [40]: `from sklearn.model_selection import train_test_split`
`X = df.drop('price', axis=1)`
`y = df['price']`
`X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_sta`

In [42]: `from sklearn.linear_model import LinearRegression`
`lm = LinearRegression()`
`model = lm.fit(X_train, y_train)`

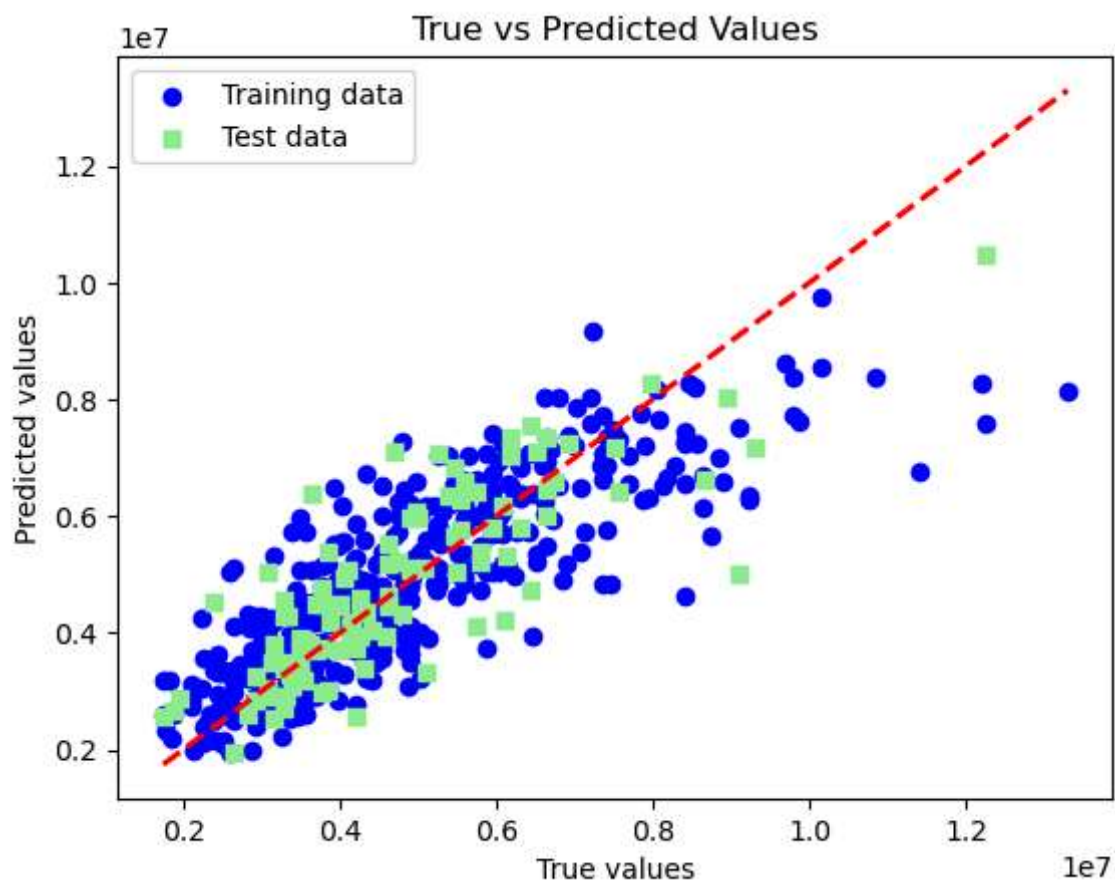
```
In [44]: ytrain_pred = lm.predict(X_train)
         ytest_pred = lm.predict(X_test)
```

```
In [45]: df=pd.DataFrame(ytrain_pred,ytrain)
         df=pd.DataFrame(ytest_pred,ytest)
```

```
In [46]: from sklearn.metrics import mean_squared_error, r2_score
         mse = mean_squared_error(ytest, ytest_pred)
         print(mse)
         mse = mean_squared_error(ytrain_pred,ytrain)
         print(mse)
```

```
976484771024.6129
1157928019785.7834
```

```
In [48]: import matplotlib.pyplot as plt
         plt.scatter(y_train, ytrain_pred, c='blue', marker='o', label='Training data')
         plt.scatter(y_test, ytest_pred, c='lightgreen', marker='s', label='Test data')
         plt.xlabel('True values')
         plt.ylabel('Predicted values')
         plt.title('True vs Predicted Values')
         plt.legend(loc='upper left')
         plt.plot([y.min(), y.max()], [y.min(), y.max()], color='red', linewidth=2, linestyle='dashed')
         plt.show()
```



```
In [ ]:
```